Laisse-moi d'abord t'expliquer le projet, puis on attaquera le TODO.

🏠 C'est quoi ID Immobilier ?

Imagine que tu veux acheter ou louer une maison à Lomé. Comment savoir si le prix qu'on te propose est juste ou exagéré ? Au Togo (comme dans beaucoup de pays africains), il n'existe pas de référence officielle claire. Les prix sont fixés "à l'œil", les annonces sont éparpillées sur Facebook, CoinAfrique, TogoImmobilier, etc.

ID Immobilier veut résoudre ce problème en construisant un système intelligent qui :

1. **Collecte** les données de plusieurs sources (annonces en ligne, agences, données de l'État comme l'OTR ou le cadastre)
2. **Nettoie et unifie** toutes ces données hétérogènes (certaines en FCFA, d'autres incomplètes, des doublons...)
3. **Calcule un prix au m²** fiable par quartier/zone
4. **Crée un indice immobilier** — c'est-à-dire un chiffre synthétique qui dit "dans ce quartier, le marché monte, descend ou est stable"
5. **Visualise tout ça** dans un tableau de bord accessible à tous

En gros, c'est un **projet Data for Public Good** : tu utilises la data pour aider les citoyens, les investisseurs et l'État à prendre de meilleures décisions sur le logement.

Et toi, tu joues le rôle d'un **Data Engineer / Data Analyst** qui construit ce pipeline de bout en bout.

En regardant tes fichiers sources, je vois que tu as déjà :

- Des données **ImmoAsk** (annonces reformatées)
- Des données **Facebook** (annonces scrappées)
- Des données **CoinAfrique / TogoImmobilier** (via Cyrille)
- Des **valeurs vénales du Togo** (données quasi-officielles)

C'est exactement le multi-source demandé par le projet. Maintenant passons au TODO.

✅ TODO — Projet ID Immobilier

📌 Phase 0 — Compréhension & Organisation

- Lire et analyser le TDR ✅
- Identifier toutes les sources de données disponibles ✅
- Définir le périmètre géographique du MVP (ex: Lomé uniquement ?)

- Constituer l'équipe et répartir les rôles
- Versionner le projet sur **GitHub** dès le départ

📌 Phase 1 — Collecte des données *(Data Ingestion)*

Technos : Apache Airflow (orchestration des pipelines), Python (requests, BeautifulSoup, openpyxl), Apache Kafka (si ingestion en flux/streaming simulé)

- [ ] Inventorier les 4 fichiers sources disponibles (Excel, CSV)
- [ ] Identifier les colonnes communes entre les sources (prix, surface, localisation, type de bien...)
- [ ] Documenter la structure de chaque source dans un fichier de mapping
- [ ] Créer un **DAG Airflow** pour automatiser la lecture et l'ingestion de chaque source à intervalles réguliers
- [ ] (Optionnel avancé) : Simuler un flux de données en temps réel avec **Apache Kafka** — chaque nouvelle annonce = un message dans un topic Kafka
- [ ] Stocker les données brutes dans une **zone Raw/Landing** (HDFS ou un dossier structuré local)

📌 Phase 2 — Nettoyage & Standardisation *(Data Cleaning)*

Technos : Apache Spark (PySpark) pour le traitement distribué, Jupyter Notebook pour l'exploration, Python/pandas pour les petits volumes

- [ ] Charger chaque fichier source avec **PySpark** (ou pandas si volume faible)
- [ ] Standardiser les noms de colonnes (snake_case unifié)
- [ ] Harmoniser les prix (tout en FCFA, même unité)
- [ ] Harmoniser les surfaces (tout en m²)
- [ ] Nettoyer les valeurs aberrantes avec des règles Spark (ex: prix < 10 000 FCFA → suspect)
- [ ] Gérer les valeurs manquantes (suppression ou imputation)
- [ ] Éliminer les doublons inter-sources avec **Spark DataFrame deduplication**
- [ ] Standardiser les noms de zones/quartiers (ex: "Adidogomé" vs "adidogome" → normalisation via dictionnaire)
- [ ] Stocker les données nettoyées dans une **zone Cleaned/Silver** (Parquet sur HDFS ou local)

📌 Phase 3 — Modélisation & Stockage *(Data Modeling)*

Technos : PostgreSQL (base relationnelle), Apache Hive (entrepôt de données sur HDFS), Draw.io pour le schéma entité-relation

- [ ] Concevoir le modèle de données (schéma entité-relation) avec au moins :

    - `bien_immobilier` (id, type, surface, prix, usage...)
    - `zone_geographique` (id, nom, commune, coordonnées GPS...)
    - `source_donnees` (id, nom, url, date collecte...)
    - `annonce` (id, id_bien, id_source, date, prix_annonce...)
- [ ] Créer les tables dans **PostgreSQL** pour les analyses relationnelles
- [ ] Créer les tables externes dans **Apache Hive** pour les analyses big data sur les fichiers Parquet
- [ ] Injecter les données nettoyées via **PySpark → PostgreSQL** (JDBC connector)

📌 Phase 4 — Calcul des indicateurs *(Data Analytics)*

Technos : Apache Spark SQL / PySpark (calculs distribués), dbt (Data Build Tool) pour transformer les données en couche analytique, PostgreSQL

- [ ] Calculer le **prix au m²** pour chaque bien (Spark SQL)
- [ ] Calculer le **prix moyen au m² par zone** (GROUP BY avec Spark ou dbt)
- [ ] Calculer la **médiane des prix** par zone et par type de bien
- [ ] Calculer l'**écart** entre prix de marché (annonces) et valeurs vénales officielles
- [ ] Détecter les zones sur/sous-évaluées
- [ ] Stocker les résultats dans une **zone Gold/Analytics** (tables agrégées dans PostgreSQL ou Hive)

📌 Phase 5 — Création de l'indice *(ID Immobilier Index)*

Technos : PySpark ou Apache Flink (si calcul en flux continu simulé), Python pour la formule

- [ ] Choisir une **période de référence** (ex: mois le plus ancien dans les données)
- [ ] Calculer l'indice : `Indice = (Prix moyen période N / Prix moyen période référence) × 100`
- [ ] Implémenter ce calcul en **PySpark** pour supporter le passage à l'échelle
- [ ] (Optionnel avancé) : Utiliser **Apache Flink** pour recalculer l'indice en quasi-temps réel à chaque nouvelle annonce ingérée via Kafka
- [ ] Calculer l'indice par zone et par type de bien
- [ ] Analyser les tendances (hausse / baisse / stabilité)

📌 Phase 6 — Visualisation & Dashboard *(Data Viz)*

Technos : Power BI ou Apache Superset (connecté à PostgreSQL), Python Folium (cartes interactives), Plotly/Dash (dashboard web léger)

- [ ] Connecter **Apache Superset** à PostgreSQL pour un dashboard open source
- [ ] Graphique d'évolution du prix moyen dans le temps

- [ ] Carte de chaleur (**Folium**) des prix au m² par quartier sur une carte de Lomé
- [ ] Bar chart comparatif par type de bien
- [ ] Tableau de bord interactif MVP avec :
  - Sélecteur de zone
  - Affichage du prix moyen au m²
  - Affichage de l'indice immobilier
  - Comparaison entre zones

📌 Phase 7 — Documentation & Présentation

- [ ] Rédiger un rapport d'analyse (méthodologie, résultats, limites)
- [ ] Schématiser l'**architecture Big Data complète** du pipeline (Lambda Architecture ou Kappa)
- [ ] Préparer une présentation claire pour le prof
- [ ] Versionner tout le code sur **GitHub**

🗺 Vue d'ensemble de l'architecture

```
Sources (Excel/Web/Kafka)
        ↓
   [Apache Airflow] ← orchestration
        ↓
   Zone RAW (HDFS / local)
        ↓
   [Apache Spark - PySpark] ← nettoyage & transformation
        ↓
   Zone CLEAN (Parquet / Hive)
        ↓
   [dbt + PostgreSQL] ← modélisation analytique
        ↓
   Zone GOLD (tables agrégées)
        ↓
   [Superset / Power BI / Dash] ← visualisation
```

Tu veux qu'on commence par explorer tes fichiers de données pour voir ce qu'on a concrètement ? Je peux les analyser et te dire ce qui est utilisable, ce qui manque et comment on peut les fusionner.